TrustTrail

Accountable, purpose-bound data release for campus single sign-on

Vijender Yadav (23BCS132) · Ansh Hardaha (23BCS016) · Sourabh (23BEC055)
Guide: Malay Kumar
B.Tech 7th semester project review · IIIT Dharwad · November 2026

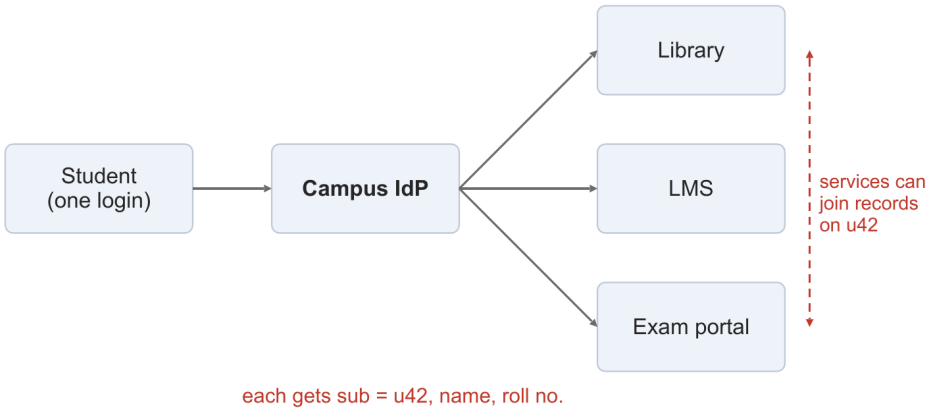
Follow-up to Werner, Westphall & Westphall, "Cloud identity management: A survey on privacy strategies," Computer Networks, 2017

Speaker 1 (Vijender) · ~30 s

Good morning. We are Vijender, Ansh and Sourabh, and our guide is Malay Kumar. Our project is TrustTrail: a privacy layer for campus single sign-on that limits what each service receives, makes every service sign a promise about why it wants the data and when it will delete it, and keeps a tamper-evident record of every release. It started as a follow-up to the 2017 survey by Werner and colleagues. I'll cover the problem, Ansh will explain how the system works and give a live demo, and Sourabh will present the implementation and results.

Slide 2 of 22 · Campus SSO gives every service the same ID and personal data, and leaves no record

Campus SSO gives every service the same ID and personal data, and leaves no record



- **Over-release:** the library gets your name and roll number
- **Linkable:** one ID lets services join their data
- **No record:** nobody can see who got what, why, or until when

GDPR Art. 5 and India's DPDP Act 2023 require purpose limitation, minimisation and accountability

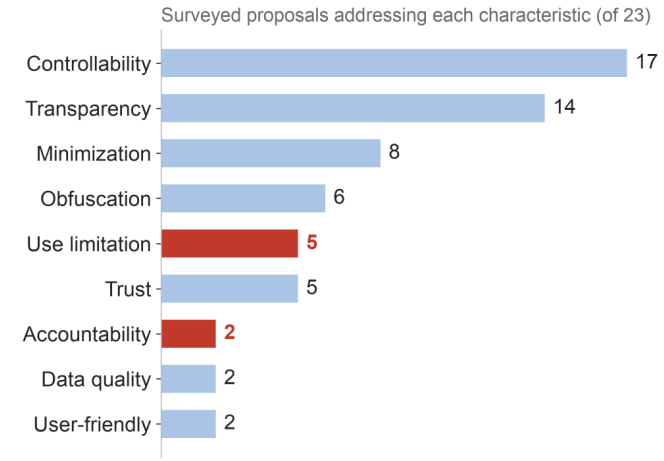
Part 1 · Problem and idea 2

Speaker 1 (Vijender) · ~1 min

Here is what a typical campus SSO setup does. You log in once, and the identity provider sends every service the same identifier and the same personal claims, whether the service needs them or not. Three problems follow. Over-release: the library only needs to know you are a student, but it gets your name and roll number. Linkability: because every service sees the same ID, any two services can join their records. And no record: you cannot find out which service received what, for which purpose, or until when. Data protection law, both GDPR and India's DPDP Act, asks for exactly the opposite.

Slide 3 of 22 · Werner et al. (2017) show accountability and use limitation are among the least-covered privacy goals

Werner et al. (2017) show accountability and use limitation are among the least-covered privacy goals



- **Accountability:** 2 of 23 proposals
- **Use limitation:** 5 of 23 proposals
- **Authors' own 4 follow-up works:** 0 for both
- Survey also names “ensure the implementation of policies” as an open challenge

Source: Werner, Westphall & Westphall, Computer Networks 122 (2017), Tables 2–3 (counts from the survey's own marks)

Part 1 · Problem and idea 3

Speaker 1 (Vijender) · ~1.5 min

We started from the 2017 survey by Werner, Westphall and Westphall. It defines nine privacy characteristics for identity management and marks which of 23 published proposals address each one. Counting their own table: accountability is addressed by only 2 of 23, and use limitation, meaning data used only for the stated purpose, by 5. The authors' own four follow-up works address neither. Most work focuses on the moment of disclosure; very little on what happens around and after it. The survey itself lists 'ensuring that policies are implemented' as an open challenge. Data quality and user-friendliness are also rare, but they matter less for our problem.

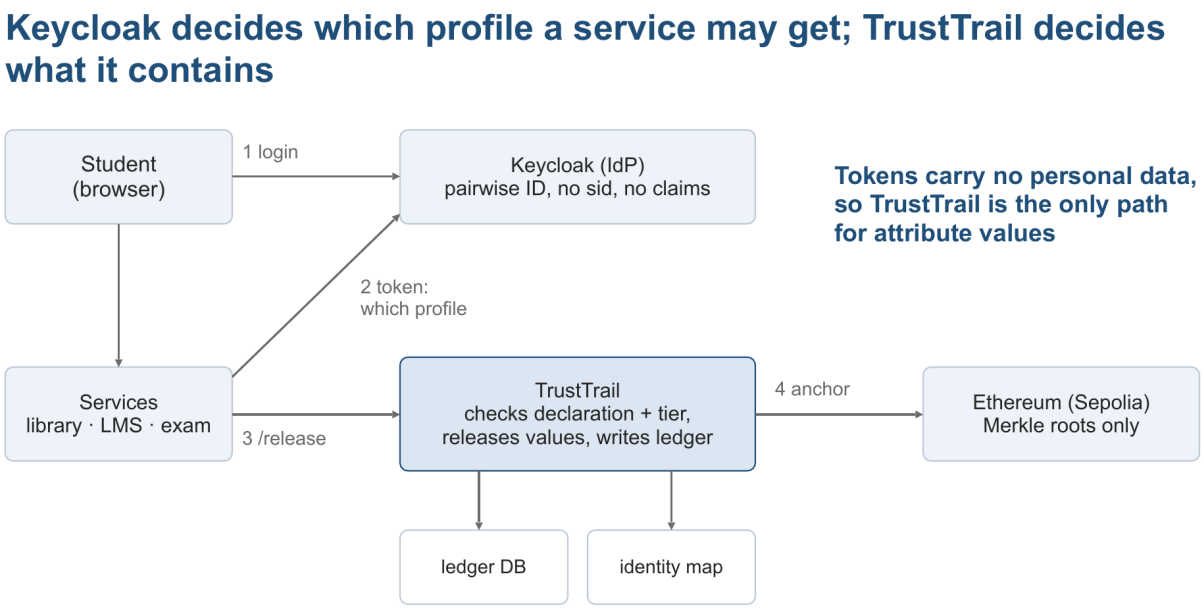
We set out to answer five questions

RQ1	Can each service get only its agreed attributes, without being able to link students across services?
RQ2	Can every release be bound to a purpose and deletion date the service has signed?
RQ3	Can the release record be tamper-evident, even against the operator, without personal data on a blockchain?
RQ4	Can observed rule-breaking automatically reduce what a service receives?
RQ5	What does this cost in login time and blockchain fees?

Speaker 1 (Vijender) · ~45 s

These gaps gave us five research questions. One: minimal disclosure and no linking across services. Two: binding every release to a signed purpose and deletion date. Three: a record of releases that nobody can secretly change, not even the operator, without putting personal data on a blockchain. Four: automatic consequences for services that break the rules. And five: what all of this costs. The rest of the talk answers these in order.

Slide 5 of 22 · Keycloak decides which profile a service may get; TrustTrail decides what it contains



Speaker 1 (Vijender) · ~1.5 min, then hand over

The key design decision is a split. Keycloak, the identity provider, decides WHICH disclosure profile a service may get: we cap it per service using Keycloak's own scope assignment. Its tokens carry a per-service pseudonym and no personal data at all. TrustTrail decides WHAT the profile contains: the service presents the student's token at the release endpoint, and TrustTrail checks the service's signed declaration and its trust tier, releases only the allowed values, and writes a ledger entry. Every few minutes TrustTrail anchors a fingerprint of the ledger on Ethereum. Because tokens carry nothing personal, TrustTrail is the only path for attribute values. Ansh will now explain each piece.

Slide 6 of 22 · Each service gets a different pseudonym, but only after we removed Keycloak's shared session ID

Each service gets a different pseudonym, but only after we removed Keycloak's shared session ID

Profile	Released	Cap of tier
anonymous	affiliation	blocked
pseudonym	+ pseudonym (sub)	untrusted
partial	+ department, programme, year	semi-trusted
total	+ name, email, roll no.	fully trusted

Same student, three services:

```
library  f266e8cf-…
LMS      4336de76-…
exam     158b3330-…
```

Finding

Keycloak 26 puts the same session ID (sid) into every service's token in one login session.

Two services comparing sid could link the student's pseudonyms.

Our 72-line Keycloak mapper removes it; a browser-level test checks it.

Part 2 · How TrustTrail works 6

Speaker 2 (Ansh) · ~1 min

We use the four disclosure profiles from Werner's group: anonymous, pseudonym, partial and total. Each service's trust tier caps the most revealing profile it can get. Keycloak gives each service a different pairwise pseudonym, so the same student, Asha, appears as three unrelated IDs. But we found that this alone did not stop linking: Keycloak 26 adds the same session ID, called sid, to every token in one login session, so two services could simply compare it. We wrote a small Keycloak extension, 72 lines of Java, that removes it, and a test that fails if it ever comes back.

A service must sign its purpose and deletion date before it receives any data

Purpose declaration (signed by the LMS, ES256)

```
{ "client_id": "lms",
  "attributes": ["sub", "affiliation",
     "department", "program", "year"],
  "purpose": "Enrol you in the courses of
     your department, program and year.",
  "retention_until": "2027-05-09",
  "nonce": "…", "iat": …, "exp": … }
```

TrustTrail refuses and records:

- **Rule 1** · attribute not in the declaration
- **Rule 2** · request after the deletion date
- **Rule 3** · more than its tier allows

Every release returns a receipt signed by TrustTrail, bound to the declaration's hash

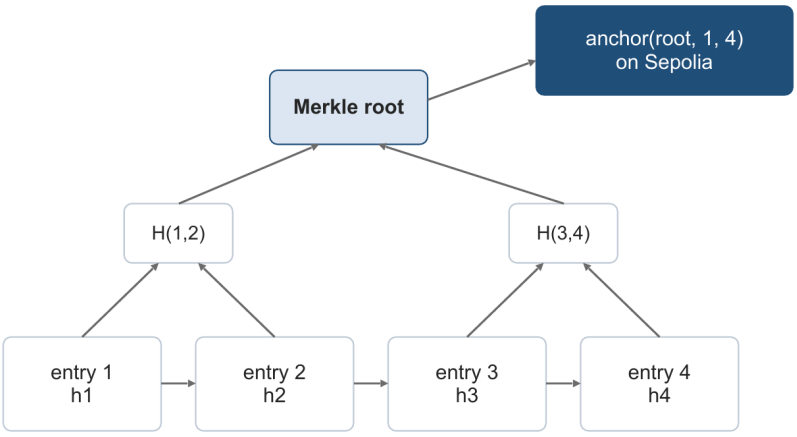
Signature formats: JWS (RFC 7515), key IDs: JWK thumbprint (RFC 7638)

Part 2 · How TrustTrail works 7

Speaker 2 (Ansh) · ~1 min

Before it gets anything, a service signs a purpose declaration with its own key: which attributes, why, and a deletion date. Here is the real declaration of our Learning System. TrustTrail stores it and refuses any request it does not cover. Three rule breaks are recorded as violations: asking for an attribute not in the declaration, asking again after the deletion date, and asking for more than the service's tier allows. Every successful release returns a receipt signed by TrustTrail, which includes the hash of the declaration. So the service cannot later deny what it promised, and the student holds proof of what was released.

Every release and rule break is hash-chained and anchored on Ethereum



each entry contains the previous entry's hash

- Only a 32-byte root goes on chain, never personal data
- Contract accepts only contiguous batches, so a cut-off ledger is visible
- Students verify their receipt against the chain without TrustTrail

Merkle hashing as in Certificate Transparency (RFC 6962); hash-chained logs: Schneier & Kelsey (1999)

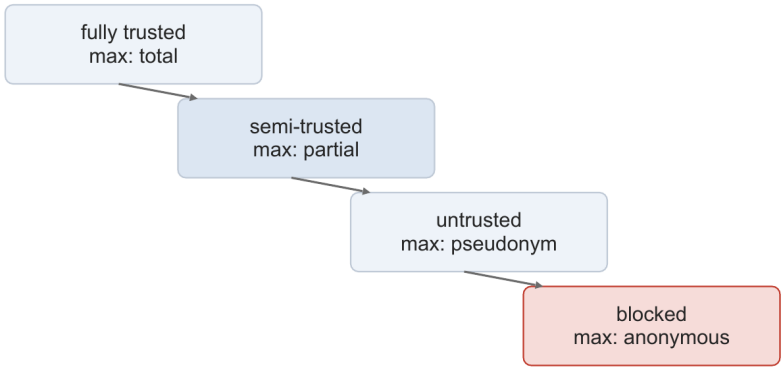
Part 2 · How TrustTrail works 8

Speaker 2 (Ansh) · ~1 min

Every release, and every refused rule break, becomes one ledger entry, and each entry includes the hash of the one before, so changing any entry breaks every link after it. But someone with the whole database could rewrite the chain from that point on. So every five minutes TrustTrail builds a Merkle tree over the new entries and stores only its 32-byte root in a smart contract on Ethereum's Sepolia test network. No personal data goes on chain. The contract only accepts batches that continue exactly where the last one ended, so a truncated ledger is visible. And a student can check their receipt against the chain without trusting our server.

Slide 9 of 22 · Breaking a rule automatically lowers what a service can receive

Breaking a rule automatically lowers what a service can receive



- Each violation costs 25 points = one tier down
- Keycloak stops granting the higher profiles; tokens already issued are capped at release
- Requests within the original agreement are not counted again, so one mistake does not snowball

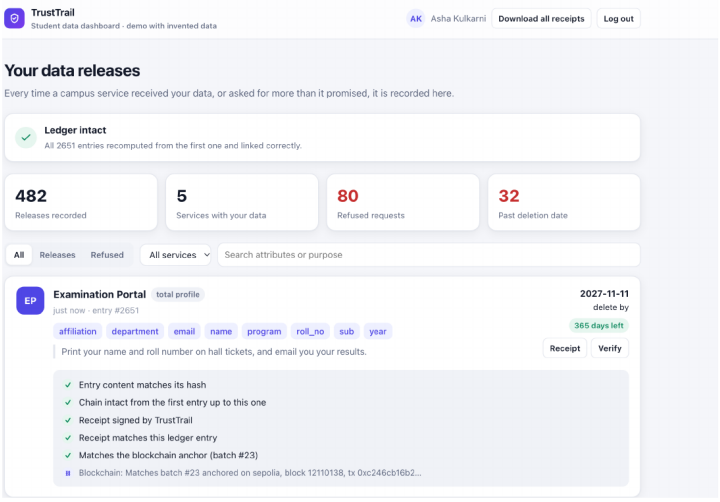
Campus Deals asked for e-mail it never declared:
semi-trusted → untrusted → blocked in two violations

Part 2 · How TrustTrail works 9

Speaker 2 (Ansh) · ~1 min

Each violation lowers the service's trust score by 25 points, which is exactly one tier down. The tier names come from Ferdous and Poet, cited in the survey. When the tier drops, TrustTrail removes the more revealing profile scopes from the service's Keycloak client, so logins asking for them fail, and tokens issued before the demotion are capped again at release. For example, our scripted Campus Deals service asked for e-mail it never declared and fell from semi-trusted to blocked in two violations. We made sure that ordinary requests within the service's agreement are not counted again, so one mistake does not snowball into automatic blocking.

Students can see, verify and download proof of every release



- Every release and refused request, with purpose and delete-by date
- Verify: hash, chain, signed receipt, and the Sepolia anchor
- Download signed receipts to check later without TrustTrail

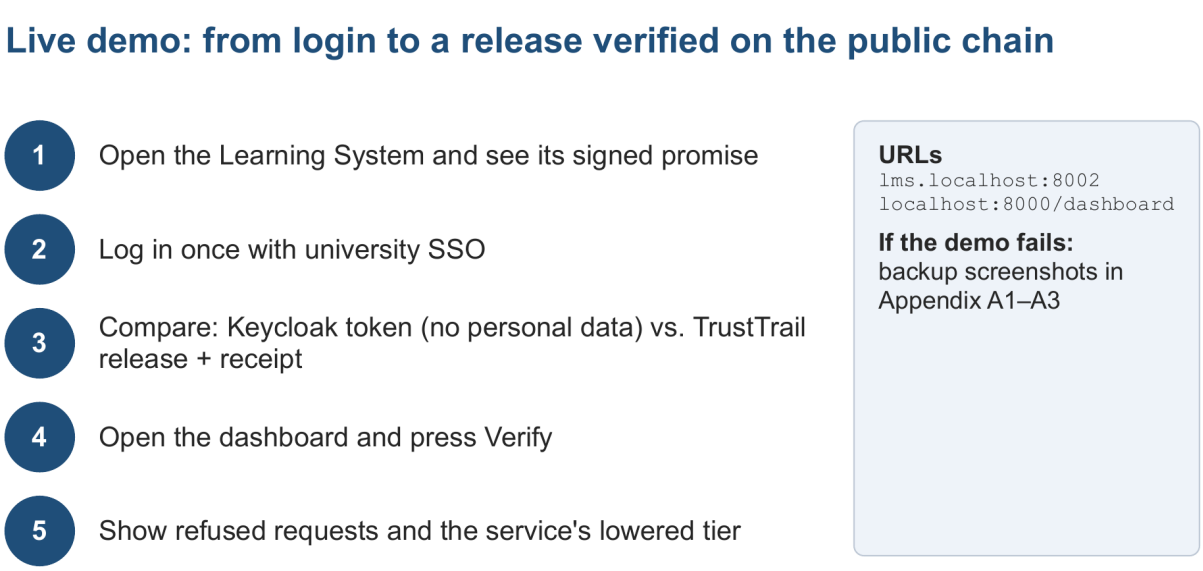
Real screenshot of our running system (invented student data)

Part 2 · How TrustTrail works

10

Speaker 2 (Ansh) · ~45 s

This is the real student dashboard from our running system, with invented data. Asha sees every release and every refused request, with the service, the attributes, the purpose and the deletion date. The Verify button runs five checks: the entry matches its hash, the chain is intact, the receipt is signed by TrustTrail, the receipt matches the entry, and the entry matches the root anchored on Sepolia. She can also download signed receipts to check later without our server. Let me show it live.



Speaker 2 (Ansh) · ~2.5 min LIVE DEMO

Before the talk: docker compose up, log out of all services, open these tabs.
Step 1: open lms.localhost:8002 and point to the signed promise: attributes, purpose, 180 days.
Step 2: log in as asha.
Step 3: show 'What this service received', then scroll to the Keycloak ID token: only a pairwise sub, no name, no sid; then the TrustTrail release and its signed receipt.
Step 4: open localhost:8000/dashboard, log in (SSO, no password), press Verify on the newest entry; all checks green, including the Sepolia anchor (if it says pending, explain the 5-minute anchoring window).
Step 5: click 'Refused' to show violations by Campus Deals and Alumni Tracker.
If anything fails, go to Appendix A1 to A3. Then hand over to Sourabh.

Slide 12 of 22 · Built on Keycloak, FastAPI and Solidity, with 151 tests, and deployed on Sepolia

Built on Keycloak, FastAPI and Solidity, with 151 tests, and deployed on Sepolia

151	~5,800	7	6
Python tests (+6 contract tests)	lines of code	Docker containers	build phases

Layer	Technology
Identity provider	Keycloak 26.8.1 + custom Java mapper
Release service	Python 3.12, FastAPI, SQLite, PyJWT (ES256 / RS256)
Blockchain	Solidity 0.8.28 contract, Hardhat 3, web3.py; Sepolia test network
Code	github.com/desmond009/Trust-Trail

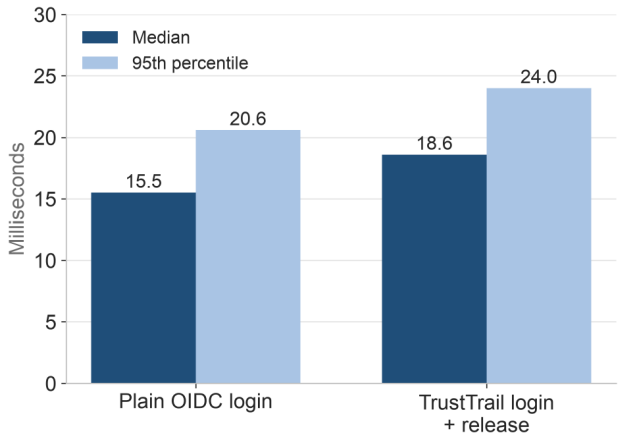
Part 3 · Implementation and results 12

Speaker 3 (Sourabh) · ~45 s

Thank you, Ansh. The system is built on Keycloak 26 with our Java extension, a Python FastAPI release service with SQLite, and a Solidity contract deployed on the Sepolia test network. It is about 5,800 lines of code in seven Docker containers, built in six phases, and covered by 151 Python tests and 6 contract tests, all passing. Everything is public on GitHub. Now the results.

Slide 13 of 22 · The privacy layer adds about 3 ms per login, mostly one Keycloak call and one commit

The privacy layer adds about 3 ms per login, mostly one Keycloak call and one commit



- **+3.2 ms median** (95% CI 3.0–3.4), 200 runs each
- Inside /release (1.2 ms): Keycloak fetch 0.50 ms, database commit 0.48 ms
- All privacy checks together: under 0.04 ms

One laptop, all containers local, password grant; the ~21% relative figure is against a 15 ms loopback login

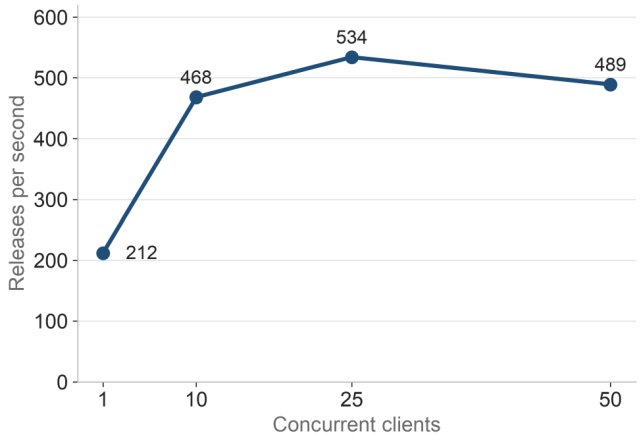
Part 3 · Implementation and results 13

Speaker 3 (Sourabh) · ~1 min

First, cost. We compared a plain Keycloak login, which gets all attributes in the token, with a TrustTrail login plus release, 200 runs each in random order. The median difference is 3.2 milliseconds, with a 95% confidence interval of 3.0 to 3.4. We then timed each step inside the release: the privacy checks themselves take under 40 microseconds; most of the time is one call to Keycloak for the student record and one durable database commit, which is the price of 'no release without a record'. The relative overhead, about 21%, sounds large only because the baseline is a 15 ms login on one machine.

Slide 14 of 22 · One server process handles about 500 releases per second before requests queue

One server process handles about 500 releases per second before requests queue



- Median latency: 2 ms (1 client) → 90 ms (50 clients)
- 0 errors in 1,180 calls; hash chain intact afterwards
- **Campus estimate:** 10,000 students × 5 services in 10 min ≈ 83/s, a sixth of capacity

Part 3 · Implementation and results 14

Speaker 3 (Sourabh) · ~1 min

Next, load. We sent release calls from 1, 10, 25 and 50 concurrent clients, every one a real release with a ledger entry. Throughput rises to a peak of about 530 releases per second at 25 clients and then flattens; after that, extra clients just wait longer, up to a median of 90 ms at 50 clients. There were no errors, and the hash chain was intact afterwards. For scale: if 10,000 students each log in to five services in the same ten minutes, that is about 83 releases per second, a sixth of what one process handles.

All six tamper attacks and 45 scripted violations were detected as designed

Attack (5 runs each)	Hash chain	Receipts*	Sepolia anchor
A. edit entry, keep hash	5/5	5/5	5/5
B. edit entry, fix hash	5/5	5/5	5/5
C. delete entry	5/5	5/5	5/5
D. rewrite all later hashes	0/5	5/5	5/5
E. cut off the tail	0/5	5/5	5/5
F. edit entry after last anchor	0/5	5/5	0/5

* assumes the affected student kept the receipt

- Every attack caught by at least one check, on local chain and Sepolia
- **Violations:** 45/45 recorded with the right rule; 0 false positives in 90 honest releases
- Conformance tests: we wrote the attacks and expected outcomes

Speaker 3 (Sourabh) · ~1.5 min

For tamper evidence we simulated an insider who can edit the database file, with six attacks, five times each, on both the local chain and Sepolia. The hash chain alone catches simple edits and deletions, but not a full rewrite from some point onward or a cut-off tail. The Sepolia anchor catches those, even if the operator does it. Only an entry made after the last anchor relies on the student's receipt. For misbehaving services, all 45 scripted violations were recorded with the right rule, and 90 honest releases produced no false alarms. We are careful to call these conformance tests: we wrote the attacks and the expected outcomes, so they show the design works as intended, not a real-world detection rate.

Slide 16 of 22 · Anchoring costs a constant 254,408 gas per batch on Sepolia, whatever its size

Anchoring costs a constant 254,408 gas per batch on Sepolia, whatever its size

Operation	Entries	Gas on Sepolia
Deploy contract	–	2,411,192
First anchor	943	349,631
Later anchors	13, 14, 16	**254,408 each**

- One 32-byte root per batch, so cost does not grow with entries
- ≈ 0.00025 ETH per batch at 1 gwei; no transaction when nothing is new
- Entry → anchor delay: median 25 s with a 30 s interval; inclusion in 6–10 s

Contract: 0x3415e1C6B20E184EA493007A56589Cebf2074BC8 (Sepolia), 10 Nov 2026. Fees are estimates for a priced network; Sepolia gas was near zero.

Part 3 · Implementation and results 16

Speaker 3 (Sourabh) · ~1 min

Finally, blockchain cost, measured on the real Sepolia network. Deployment was about 2.4 million gas once. After the first batch, every anchor costs exactly 254,408 gas, whether it covers 13 or 16 entries, because only one root is stored. At one gwei that is about 0.00025 ETH per batch, and nothing is sent when there are no new entries. With a 30-second interval, an entry was anchored after a median of 25 seconds; transactions were included in a block within 6 to 10 seconds. Anyone can inspect the contract on Etherscan.

TrustTrail gives evidence, not enforcement, after data leaves

Limits

- Cannot prove a service deleted data
- Operator decides what is written; anchors protect only anchored history
- Released attributes (department, year) can still link students
- Trust weights are heuristic; tests on one laptop

Next steps

- Usability study (designed, not yet run; ethics approval first)
- Ledger retention and erasure (DPDP Act s. 8(7), 12)
- Main network or witness cosigning
- Student choice per attribute at release

Speaker 3 (Sourabh) · ~45 s

We want to be clear about the limits. TrustTrail cannot see what a service does after it has the data, so it cannot prove deletion; it gives evidence of what was promised and received. The operator still decides what gets written, and the anchor protects only history that has been anchored. Attribute values like department and year can still link students in a small class. And our trust weights are heuristic. Next, we plan a usability study after ethics approval, a retention and erasure design for the ledger under the DPDP Act, anchoring on a main network or with independent witnesses, and letting students choose attributes per release.

Conclusions: accountable data release is practical on a standard identity provider

RQ1	Different pseudonym per service, no shared session ID, only profile attributes
RQ2	No release without a service-signed purpose and deletion date
RQ3	Anchored history is tamper-evident on a public chain, even against the operator
RQ4	Violations lower the tier automatically, with no false positives in our tests
RQ5	~3 ms per login, ~500 releases/s, 254,408 gas per batch

github.com/desmond009/Trust-Trail

Vijender Yadav · Ansh Hardaha · Sourabh
IIIT Dharwad

Thank you — questions welcome

18

Speaker 3 (Sourabh) · ~45 s, then Q&A; (leave this slide up)

To conclude, our five questions. One: each service gets a different pseudonym and only its profile's attributes, with no shared session ID. Two: no data is released without a purpose and deletion date the service has signed. Three: anchored history is tamper-evident on a public chain, even against the operator. Four: violations automatically lower a service's tier. Five: it costs about 3 ms per login, handles about 500 releases per second, and 254 thousand gas per anchor. The code is public at this QR code. Thank you; we are happy to take questions.

References

J. Werner, C. M. Westphall, C. B. Westphall, “Cloud identity management: A survey on privacy strategies,” Computer Networks, vol. 122, pp. 29–42, 2017.

N. Sakimura et al., “OpenID Connect Core 1.0,” OpenID Foundation (pairwise identifiers, Sec. 8).

M. Jones, J. Bradley, N. Sakimura, “JSON Web Signature (JWS),” RFC 7515, 2015; M. Jones, N. Sakimura, “JWK Thumbprint,” RFC 7638, 2015.

B. Laurie, A. Langley, E. Kasper, “Certificate Transparency,” RFC 6962, 2013.

B. Schneier, J. Kelsey, “Secure audit logs to support computer forensics,” ACM TISSEC, vol. 2, no. 2, 1999.

S. A. Crosby, D. S. Wallach, “Efficient data structures for tamper-evident logging,” USENIX Security, 2009.

M. S. Ferdous, R. Poet, “Managing dynamic identity federations using SAML,” J. Theor. Appl. Electron. Commer. Res., 2015.

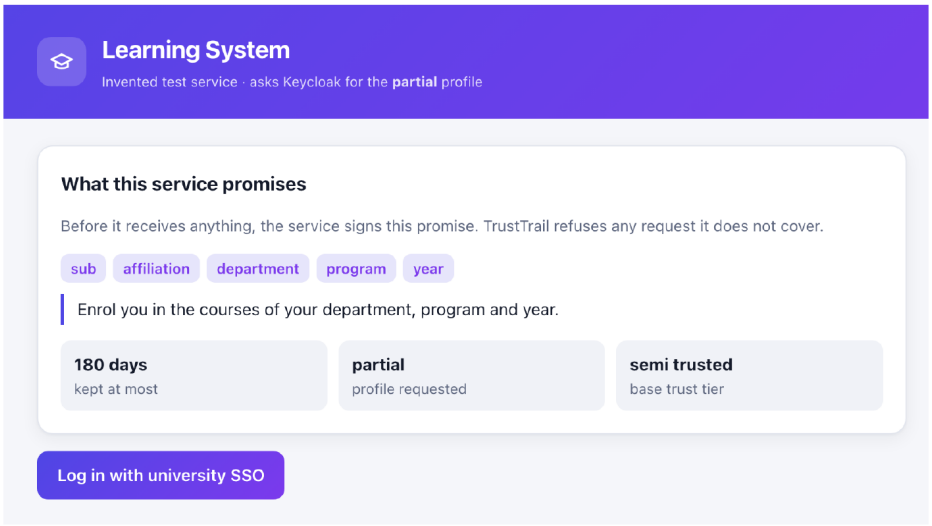
Regulation (EU) 2016/679 (GDPR); The Digital Personal Data Protection Act, 2023 (India).

Full bibliography (69 references): TrustTrail paper, github.com/desmond009/Trust-Trail

Presenter notes

References for the slides. The paper has the full verified bibliography.

A1 · Before login, the Learning System shows the promise it has signed

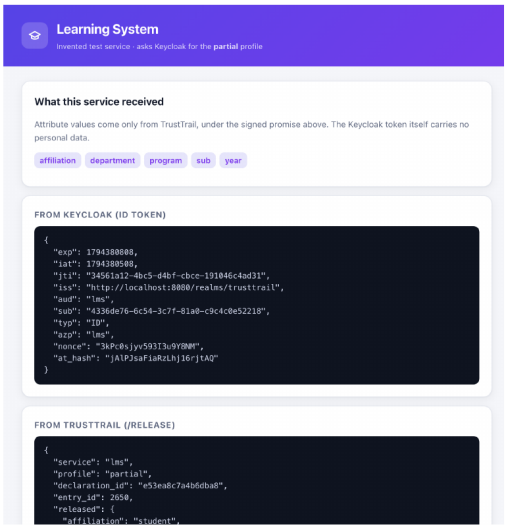


Presenter notes

Backup for demo step 1. The service shows its signed declaration: five attributes, the purpose, kept at most 180 days, partial profile, semi-trusted tier.

Slide 21 of 22 · A2 · After login, attribute values come only from TrustTrail; the Keycloak token has none

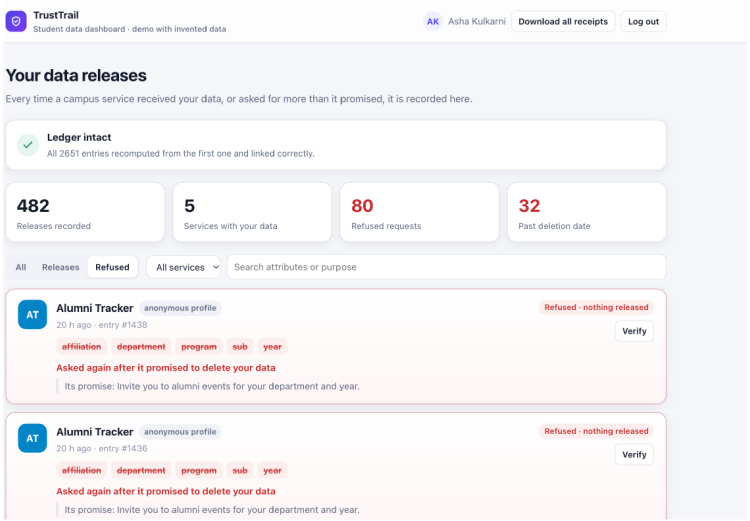
A2 · After login, attribute values come only from TrustTrail; the Keycloak token has none



Presenter notes

Backup for demo step 3. Top: what the LMS received from TrustTrail. Below: the Keycloak ID token, with only a pairwise sub (4336de76...) and no name, e-mail, roll number or sid.

A3 · Refused requests appear on the dashboard and lower the service's trust



Appendix · backup demo screenshots 22

Presenter notes

Backup for demo step 5. Refused requests by Alumni Tracker, which asked again after its promised deletion date; nothing was released and each is a ledger entry. The Sepolia contract can be shown live on sepolia.etherscan.io at address 0x3415e1C6B20E184EA493007A56589Cebf2074BC8.